ConPro PMIS - RAG Integration Implementation Plan

System-wide PMIS enhancement | Review only

2026-10-15

ConPro PMIS - RAG Integration Implementation Plan

1. Decision summary and boundaries
2. Non-negotiable acceptance invariants
3. Contract inventory and authentication wiring
4. Source eligibility, identity, and critical policy code
5. Query orchestration, disclosure, and citations
6. Authorized retrieval and hybrid prerequisites
7. Synchronization, ordering, and atomic publication
8. HTTP client, errors, limits, and capacity
9. Audit, deployment, rollout, and recovery
10. Acceptance tests engineers must implement
11. Bottleneck register and engineering resolutions
12. Manageable implementation layout and coding rules
13. Assumptions, limitations, and future recommendations
14. Review decisions and approval checklist

Appendix A. Verified source map

Appendix B. Deliverable and verification scope

ConPro PMIS - RAG Integration Implementation Plan

Date: 2026-10-15
Document version: 1.0
Status: **REVIEW COPY - NOT APPROVED FOR IMPLEMENTATION**

Purpose: **System-wide, permission-aware AI enhancement across ConPro PMIS.**

This deliverable is documentation only. The code below is proposed reference code, not installed application code. Do not run migrations, add routes, change permissions, enable providers, or deploy this integration until its implementation batch has been reviewed and explicitly approved.

Read with `CONPRO_PMIS_RAG_INTEGRATION.md` and `RAG_HYBRID_IMPLEMENTATION_PLAN.md`. Those papers describe the intended architecture. This plan specifies implementation boundaries, critical code, failure behavior, engineering tasks, and evidence required for approval.

1. Decision summary and boundaries

The objective is system-wide ConPro PMIS enhancement, not a Scratch-pad integration. Establish a shared permission-aware AI capability that existing modules can use for knowledge retrieval, evidence-backed explanations, cross-module investigation, approved live-fact interpretation, and later human-reviewed drafting. Scratch-pad is one optional consumer, not the entry point, architectural center, or limit of the program.

Implement an opt-in, read-only ConPro adapter inside the existing Django backend, calling a separately authenticated integration boundary in the existing RAG FastAPI process. Keep ConPro's existing login, records, workflows, reporting, protected downloads, and UI. Keep the existing personal-key RAG application.

ConPro remains the authority for **who may use which source, which revision is current, and whether external processing is permitted**. RAG supplies bounded retrieval and evidence-grounded generation, not PMIS permissions or business decisions.

The first approved pilot should cover one project and at most 20 explicitly selected controlled documents. Admit only normal-confidentiality, current, eligible, supported document versions. Exclude attachments, incidents, private sections, structured financial records, arbitrary URLs/SQL, portfolio questions, historical comparison, and autonomous actions until separately approved. These exclusions bound the first validation release; they do not remove other PMIS modules from the system-wide implementation roadmap below.

Recommended production-capable pilot storage is pgvector with PostgreSQL full-text search in a separate RAG-owned database/schema. This does not authorize access to ConPro business tables. Independent lexical retrieval and RRF are **not yet implemented**; they remain a separate dependency and release gate.

1.1 Two implementation lanes

Lane	Purpose	Release condition
Integration foundation	Identity, source identity, safe synchronization, scoped dense retrieval, structured evidence.	May be reviewed and tested independently, with hybrid explicitly disabled.
Hybrid integration	Independent dense + lexical retrieval, identical eligibility, bounded identity-based RRF.	Requires the hybrid plan's tests and representative quality evaluation.

Do not make an unavailable hybrid implementation look successful by silently falling back to dense mode. A dense-only pilot must be explicitly selected and approved as dense-only. Neither lane is authorized by producing this paper.

1.2 Verified baseline; do not reimplement finished work

Application	Inspected source baseline	Existing capabilities
RAG	Branch `rag`, commit `f0f3e6d`.	Per-user hashed keys and exact project grants; approved database sources; safe staging; scoped jobs/feedback; stable IDs; atomic replacement; corrected caches; bounded concurrent questions.
ConPro	Branch `conpro-working`, commit `64ae8ba`, with existing unrelated working-tree changes.	Django/DRF, audited JWT/session authentication, project scoping, published-role policy, controlled documents/versions, authenticated downloads, audit, existing optional AI hooks.

The earlier integration/hybrid papers reference an older RAG snapshot. Their statements about anonymous RAG access and missing ID/cache/staging fixes are historical, not the current baseline. Their integration-specific gaps remain: record/version allowlists, safe field projections, durable synchronization, structured citations, and ConPro policy checks around provider dispatch.

The previous code-change validation reported 139 Python tests passed, two PostgreSQL tests skipped, 15 frontend tests passed, and a successful frontend build. Those results are historical evidence for that change, **not integration tests**. No deployed ConPro policy, live database, real source file, credential, or provider configuration was inspected for this document.

1.3 Architecture that stays, infrastructure that may change later

```
Existing ConPro browser + existing authentication
    -> existing Django API + proposed RAG adapter
       -> live ConPro authorization / opaque query scope
       -> private integration API in existing FastAPI process
          -> authorized source-version set
          -> consistent PostgreSQL retrieval snapshot
          -> optional independent dense + lexical retrieval / RRF
          -> dispatch permission check
          -> approved provider
          -> evidence validation + final permission/freshness checks
       -> ConPro audit + trusted source descriptors
    <- answer and protected version-specific download actions

Separate indexing path:
ConPro document mutation + durable outbox in one transaction
    -> bounded existing-deployment worker / management command
    -> validated file + immutable source/version manifest
    -> staged parse/chunk/embed/index
    -> atomic RAG publication + durable acknowledgement
```

No new standalone application, general agent framework, browser-to-RAG login, message broker, Kubernetes cluster, or multi-region deployment is required for the first pilot. A database-backed outbox/worker is sufficient only after its recovery and delivery guarantees have been tested.

1.4 System-wide capability and module coverage

Provide one shared integration boundary and policy/evidence contract, consumed by existing project and module screens. Do not build disconnected per-module chatbots or force users through Scratch-pad.

ConPro area	Proposed enhancement and evidence	Required boundary
Projects, contracts and project controls	Project briefings; trace requirements, obligations, milestones and issues to authorized records/documents.	Project/company policy; exact budget/cost figures from existing calculations, not vector similarity.
Document Control	Find and explain requirements; revision-specific evidence; later explicit authorized revision comparison.	Current/effective status, confidentiality, immutable versions and protected downloads.
RFIs	Explain questions/responses; locate related specifications, changes and schedule references; prepare suggested response text later.	RFI stage/record/company policy; drafts are never official responses.
Submittals	Summarize permitted review history; locate governing requirements and unresolved comments.	Review/revision/status rules; no automatic approval or issue action.
Changes and commercial administration	Trace permitted change reasons and supporting RFI/document evidence; summarize approved history.	Commercial confidentiality; official valuation and entitlement decisions remain with ConPro/authorized people.
Daily Reports / IDR	Summarize authorized site observations, activities and evidence; explain trends using approved facts.	Inspector/manager visibility, draft/locked rules, attachment and personal-data restrictions.
Schedule	Explain authorized activity/dependency/milestone status and cited causes of delay.	Dates, critical path and delay calculations from approved schedule/reporting logic, not LLM arithmetic.
Payments, cash flow and final cost	Explain authorized certificates, deductions and variances with current factual receipts.	Amounts, currency, certification, payment and forecasts remain authoritative ConPro calculations/workflows.
Quality / QC	Explain permitted findings, inspection evidence and recurring issues; connect to specifications/RFIs.	Contractor-company/draft visibility and existing QC rule results; AI cannot close a finding.
Incidents / NCRs	Later safe summaries of authorized non-restricted facts and corrective-action evidence.	Incident row/duty/field policy; personal, medical, HR and privileged sections excluded unless separately approved.
Reporting / executive dashboards	Evidence-linked explanations combining approved report facts and source records.	Preserve existing numeric/reference checks and deterministic figures; do not replace existing executive AI silently.
Attachments	Supporting evidence across modules, not a permission-independent document pool.	Parent record's visibility, source type, immutable content digest and protected source action.
Spreadsheets	Later explain approved report/cell snapshots and linked project evidence.	Allowlisted ranges/formulas/versions only; no arbitrary workbook execution, macros or invented totals.
Custom fields	Add approved business context to module-specific projections.	Versioned field schema and field permissions; never serialize every `custom` field automatically.

ConPro area	Proposed enhancement and evidence	Required boundary
Accounts, organization, ethics and audit	Approved policy/process guidance and permitted compliance explanations.	Never index credentials, tokens, unrestricted user directories or raw security/audit logs by default.
Scratch-pad	Optional evidence-backed note assistance and draft organization.	Private-note ownership and existing behavior; not the sole integration surface.

This matrix describes proposed product coverage, not proof that existing hooks already provide these capabilities. Each module needs its own approved projection, lifecycle mapping, policy adapter, evidence navigation, and acceptance tests before it is enabled.

1.5 Three evidence paths, not one text index for everything

Path	Appropriate work	Implementation rule
Document evidence	Specifications, controlled files, permitted attachments and historical text.	Versioned extraction, dense/lexical retrieval, exact provenance.
Record evidence	RFI responses, submittal history, permitted quality/daily-report/change narratives.	Allowlisted, versioned safe projections exported by the owning ConPro module.
Live facts	Current payment totals, schedule indicators, costs, counts and report metrics.	Typed read-only calls to approved ConPro calculation/reporting functions; dated factual receipts.

An answer may combine the three paths, but must distinguish “current calculated fact as of …” from “historical source states …”. An indexed amount is not a current ledger balance. A model does not perform contractual, financial, schedule, or engineering calculations on ConPro’s behalf.

Use a small server-owned catalog of approved question/fact operations, not generated SQL or a model-selected arbitrary URL/function. Ambiguous requests require clarification or abstention. Cross-module joins follow existing authorized business relationships; they must not reveal a linked record that the user cannot otherwise access. Cross-project/portfolio access requires its own authorization gate.

2. Non-negotiable acceptance invariants

1. Feature off leaves both existing applications’ behavior unchanged.
2. Browser requests never choose the integration namespace, user identity, authorization manifest, provider, or arbitrary callback/download URL.
3. Project membership alone is not document authorization. Apply active-user, ethics, project, module, company, record, confidentiality, version, and processor/export policy.
4. Resolve eligibility before candidate scoring, ranking, counts, snippets, or provider context. Post-filtering the final top five is insufficient.
5. Current means an explicitly eligible current revision, not simply the newest file ever uploaded or a previously indexed approved file.
6. Recheck permission and freshness immediately before provider dispatch, before RAG returns, and before ConPro releases the response.
7. A lost deletion/sync event must not grant query access. Live policy remains authoritative while derived data is being removed.

8. File versions, hashes, projections, index profiles, and genuine chunk IDs remain attached to evidence. Identical text from different records is not the same source.
9. Never hold a source/database/index transaction open during provider calls.
10. "Accepted" or "queued" never means "indexed and ready."
11. No unfiltered retry, silent topology/mode fallback, raw internal error, arbitrary SQL, workflow mutation, or external-knowledge bypass.
12. No scaling claim without representative load, isolation, failure, and cost measurements.

3. Contract inventory and authentication wiring

Everything named **proposed** below is absent until an approved implementation creates and tests it.

3.1 Browser-facing ConPro endpoints - proposed

Endpoint	Caller and contract	Required result
`POST /api/rag/ask/`	Existing authenticated ConPro user; `{project, question}`.	Structured answer or explicit safe status; no RAG/service credentials.
`GET /api/rag/status/?project=<id>`	Existing user with project/document visibility.	Authorized readiness summary only; no hidden document counts.
`POST /api/rag/sync/`	Explicit project integration-operator permission.	Durable operation ID and `queued`, not immediate readiness.

Project omission, unknown fields that attempt to set authority, overlong/blank questions, and malformed bodies must fail validation. A missing or invisible project has the same 404 behavior. A visible project without eligible evidence returns `no_answer` or `index_not_ready`, not another project's information.

Do not directly add this POST to a generic ViewSet and assume `RoleScopedPermission` treats it as a read: the existing `view_action` maps an unrecognized POST to `edit`. Use an APIView with explicit read/export policy, retaining the normal authentication and ethics gates.

3.2 Private RAG boundary - proposed

Use the namespace `/integrations/conpro/v1/`:

Proposed operation	Required authority
`POST .../ask`	Registered integration query client + live ConPro query scope.
`POST .../source-operations`	Separately authorized ingestion client; one bounded manifest/file operation.
`GET .../operations/{operation_id}`	Same registered client and namespace ownership.
`GET .../capabilities`	Registered client; contract/backend/profile/mode/limit metadata only.

The source operation is multipart for a file plus a validated JSON manifest, or a metadata-only tombstone. Rebuild/replay is an operator operation, not a browser file URL. Split query and ingestion credentials/permissions, even if both are initially managed by one deployment team.

Critical current incompatibility: `app.main` has global personal-key dependencies and pre-body middleware. The existing `RAG_SERVICE_TOKEN` protects retrieval/inference routes; it does not authenticate calls to main `/ask`. Adding only a new route-level dependency cannot bypass or replace that middleware.

Proposed wiring must be reviewed as one coherent change:

1. Mount a small integration FastAPI sub-application at the exact namespace. Parent FastAPI route dependencies are not a substitute for child dependencies.
2. Extend the existing pre-body middleware to authenticate registered integration credentials for that exact mount path before forwarding. Retain personal-key authentication for every existing protected route.
3. The child application independently requires the verified integration principal and operation permission. A service credential is never a user authorization manifest.
4. Match the prefix with a path-segment boundary. Test lookalike prefixes, encoded paths, trailing slashes, redirects, OPTIONS, and unknown child routes.
5. Apply request-size limits before multipart parsing and actual streamed-byte limits during file copy. Reject, rather than redirect, authenticated calls.
6. Validate required registries, TLS trust, callback origin, profile, and backend at startup when enabled. Missing configuration fails closed.
7. Share one process-level answer admission limiter between personal `/ask` and integration `/ask`. Do not create a separate four-slot pool for each.

No implementation should exempt the entire `/integrations` prefix anonymously or use the local demo account as a connector identity.

3.3 ConPro private policy operations - proposed

Operation	Purpose
`POST /api/internal/rag/scopes/resolve/`	Resolve a query handle to a bounded authorized source manifest.
`POST /api/internal/rag/scopes/check/`	Recheck subject, scope, selected sources, expiry, provider approval, and requested phase.
Internal source-operation verification	Confirm latest source sequence and indexing/processor eligibility before publication.

Authenticate the registered RAG service, not a browser subject, at the private route. Separately reload and authorize the subject named in the stored scope. Do not accidentally apply a service account's ethics/record permissions in place of the real user's. Never expose these routes to the public frontend.

3.4 Recommended transport and scope design

Use private TLS with certificate validation; require mTLS for production where the ingress supports it, plus separately rotatable registered service credentials. Use the organization's established certificate/secret management rather than writing a new signing system. Non-loopback plaintext transport is not acceptable.

Recommend a **server-side opaque scope handle**, not a custom JWT format:

- Generate at least 256 random bits using the platform's cryptographic generator.
- Store only its digest in ConPro; transmit the handle only in authenticated request bodies, never URLs, logs, audit comments, or browser storage.
- Bind it to subject, request UUID, canonical question digest, project, deployment namespace, allowed source manifest, processor, retrieval mode, and index profile.

- Proposed maximum lifetime: 90 seconds, also bounded by the authenticated session/access-token lifetime. Every check rejects expired, inactive, revoked, mismatched, or unknown scopes. Enforce total-request deadlines separately.
- Validate a fixed phase transition: issued -> resolved -> dispatch-approved -> completed/denied/expired. Bind checks to the same registered service/request. Duplicate checks are idempotent only for identical payloads and legal phases.
- Recompute authoritative eligibility, not just handle existence. For the pilot, any manifest/policy fingerprint change invalidates the pending query.
- A handle contains no general read privilege and cannot authorize a different question or operation. Delete expired state under the approved retention policy.

Access-token refresh revocation is not necessarily immediate revocation of every already-issued access token. Do not claim otherwise. Account disablement, permission expiry, ethics status, document policy, and any required explicit session revocation must be checked by the policy adapter.

4. Source eligibility, identity, and critical policy code

4.1 Exact pilot corpus policy - recommended for review

Dimension	Proposed initial rule
Scope	One approved deployment namespace and pilot project; at most 20 selected documents.
Confidentiality	`NORMAL` only. Exclude `CONFIDENTIAL` and `RESTRICTED`, even for privileged users.
Status	`APPROVED` or `ISSUED_FOR_CONSTRUCTION` only. Adding other statuses needs domain-owner approval.
Revision	`Document.current_version` must identify the eligible version; new draft revision suppresses the older approved version.
Dates	Future effective dates excluded; expired documents excluded. Missing effective date is acceptable only with the approved status. Use ConPro's configured timezone for the pilot.
Format	Text-bearing PDF, DOCX, TXT first; images only after OCR deployment/quality approval.
Rights	Existing document/version visibility plus applicable `view` and `export` rights and a separate approved processor policy.
Coverage	Every source in an issued scope must be ready at its exact hash/profile; otherwise `index_not_ready`. No silent partial-corpus answer.

`APPROVED_WITH_COMMENTS`, `ISSUED_FOR_INFORMATION`, records with unresolved review conditions, historical versions, drawings requiring geometry interpretation, and scanned-PDF OCR are deliberately not automatic pilot admissions.

`current_version` is not proof of contractual effectiveness. Have the document controller approve the eligibility table and date semantics before implementation.

4.2 Shared authoritative policy helper - proposed reference fragment

Suggested home: `ConPro\backend\apps\documents\visibility.py` for extracted existing visibility, with integration-specific policy in `apps\rag\policy.py`. Do not permanently import private `_visible_documents` from a view module.

```python
from django.http import Http404
from django.utils import timezone

from apps.accounts.role_policy import allows, filter_records, module_for
from apps.accounts.scoping import require_project_access, scope_to_projects
from apps.documents.models import Confidentiality, DocStatus, Document
from apps.documents.visibility import visible_documents  # proposed extraction


def eligible_versions(user, project_id, pilot_document_ids):
    if not user.is_authenticated or not user.is_active:
        raise Http404("Not found.")
    require_project_access(user, project_id)
    today = timezone.localdate()
    docs = scope_to_projects(
        visible_documents(user), user, "folder__project_id"
    ).filter(
        pk__in=pilot_document_ids,
        folder__project_id=project_id,
        confidentiality=Confidentiality.NORMAL,
        status__in=(DocStatus.APPROVED, DocStatus.ISSUED_FOR_CONSTRUCTION),
        current_version__isnull=False,
    ).select_related("folder", "current_version")
    docs = docs.exclude(effective_date__gt=today)
    docs = docs.exclude(expiration_date__lt=today)
    if user.role_catalog_enabled:
        module = module_for(Document, "documents")
        for action in ("view", "export"):
            docs = filter_records(
                user, docs, module, project_path="folder__project_id",
                action=action,
            )
    versions = []
    for doc in docs.order_by("pk"):
        version = doc.current_version
        if user.role_catalog_enabled:
            objects = (doc, version)
            if not all(
                allows(user, project_id, module_for(type(obj), "documents"),
                       action, obj)
                for obj in objects for action in ("view", "export")
            ):
                continue
        versions.append(version)
    return versions
```

Boundary of this fragment: extraction must preserve existing folder/document confidentiality behavior, while intersecting active project scoping. Published-role `filter_records` is a bulk narrowing step, not a substitute for `allows` object checks. An organization manager's project oversight does not automatically grant the document visibility excluded by the existing document helper.

The caller must additionally enforce ethics acknowledgement, selected-document configuration, file support, source content digest, processor/export approval, and scope limits. In legacy-role mode, explicitly configure external-processing approval; do not pretend the published-role export policy was consulted. `view` alone is deliberately insufficient for sending content to an external AI.

This simple fragment favors correctness over query efficiency. Existing role functions may query assignments per object. Profile query counts; build a request-local policy context only if equivalence tests prove it matches existing policy. Never cache authorization across requests to hide an N+1 problem.

4.3 Canonical source identity

Use an operator-assigned deployment namespace, for example `conpro:<deployment-uuid>`, and a project collection `project:<project-pk>`. Organization/company, user ID, email, display project code, and filename are not substitutes for that deployment namespace.

For the pilot, the immutable source tuple is:

```
namespace + project_id + document_id + document_version_id
    + content_sha256 + projection_version + index_profile
```

`content_sha256` is calculated from actual bounded file bytes, not a browser checksum or modification timestamp. `projection_version` defines exactly what metadata/text may enter the corpus. `index_profile` pins the actual embedding model/dimension, parser version, chunking, lexical configuration, and locator semantics. Changing any of those requires a new profile/reindex gate.

Keep document identity distinct from version identity. Rename is not a new document; a revised blob is not the same evidence. Preserve separate source associations when two documents contain identical text.

4.4 Strict integration contracts - proposed reference fragment

Suggested RAG home: `app\conpro\contracts.py`. These models intentionally do not extend the legacy public `QuestionRequest` into a general permission language.

```python
from typing import Literal
from uuid import UUID

from pydantic import BaseModel, ConfigDict, Field, model_validator


class SourceRef(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)
    kind: Literal["document_version"] = "document_version"
    document_id: int = Field(strict=True, gt=0)
    version_id: int = Field(strict=True, gt=0)
    content_sha256: str = Field(pattern=r"^[a-f0-9]{64}$")
    projection_version: Literal["document-text-v1"]
    index_profile: str = Field(min_length=1, max_length=80)


class IntegrationAsk(BaseModel):
    model_config = ConfigDict(extra="forbid")
    request_id: UUID
    scope_handle: str = Field(min_length=32, max_length=128)
    question: str = Field(min_length=1, max_length=8000)


class AuthorizedScope(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)
    request_id: UUID
    question_sha256: str = Field(pattern=r"^[a-f0-9]{64}$")
    namespace: str = Field(min_length=1, max_length=128)
    project_id: int = Field(strict=True, gt=0)
    scope_digest: str = Field(pattern=r"^[a-f0-9]{64}$")
    provider_id: str = Field(min_length=1, max_length=80)
    retrieval_mode: Literal["dense", "hybrid"]
    sources: tuple[SourceRef, ...] = Field(min_length=1, max_length=200)
    expires_at_epoch: int = Field(strict=True, gt=0)
```

```
    @model_validator(mode="after")
    def unique_sources(self):
        if len(set(self.sources)) != len(self.sources):
            raise ValueError("Duplicate authorized source.")
        return self
```

The callback receiver also checks exact field lengths, canonical namespace syntax, registered client-to-namespace mapping, question digest/request binding, expiry, source-profile consistency, and the server-supported mode. Pydantic shape validation is not authorization. Canonicalize questions using a specified versioned rule, recommended NFC normalization plus outer whitespace stripping, and hash their UTF-8 bytes identically in both apps.

The hard contract cap of 200 references is a safety ceiling, **not approval to expand the 20-document pilot**. Exceeding it returns `scope_too_large`; never silently take the first 200. Larger deployments need an approved server-side authorized-set representation and backend performance tests.

5. Query orchestration, disclosure, and citations

5.1 End-to-end sequence

1. Browser calls ConPro with its existing JWT/session and selected project.
2. ConPro validates feature flag, question, active identity/ethics, project, document/version visibility, and processor approval.
3. ConPro creates a request-bound scope over current eligible pilot sources. No eligible sources means a safe local `no_answer`, without provider use.
4. ConPro calls the fixed private RAG endpoint using its query credential.
5. RAG authenticates before processing, acquires the shared nonblocking answer slot, resolves the handle through the fixed ConPro callback, and verifies namespace, binding, limits, profile, and capabilities.
6. In one short consistent database snapshot, RAG checks exact source readiness, captures corpus generation, retrieves only authorized candidates, fuses if approved, and creates an immutable CTX-to-source ledger.
7. End the retrieval transaction. Before sending text to an AI provider, perform the dispatch policy callback and a fresh generation check.
8. Call only the approved processor. Parse bounded structured output; validate every citation against the retained ledger.
9. RAG rechecks policy and generation before returning. On change, discard the answer and return an explicit stale/denied status.
10. ConPro independently rechecks current subject/source/processor policy, validates typed source descriptors, constructs trusted version-download actions, persists a minimal audit event, and returns to its existing UI.

Each cache hit would need equivalent checks. **Disable cross-request answer and retrieval caching for integration v1** to reduce authority and invalidation complexity; preserve existing legacy caching outside this boundary. Later caching needs a separately approved key/invalidation contract including scope, policy, source manifest, shared corpus generation, profile, mode, question, and provider.

5.2 Critical orchestration - proposed reference fragment

Suggested RAG home: `app\conpro\query.py`. `policy`, `search`, and `generator` below are small explicit collaborators, not a general plugin framework.

```python
def answer_integration(request, principal, policy, search, generator):
    scope = policy.resolve(
        handle=request.scope_handle,
        request_id=request.request_id,
        question=request.question,
        principal=principal,
    )
    # resolve verifies client/namespace, binding, expiry and live rights.
    evidence = search.retrieve_authorized(scope, request.question)
    # evidence contains generation, exact sources and immutable CTX blocks.
    if evidence.blocks:
        policy.check(
            scope, phase="dispatch", selected_sources=evidence.sources,
            request_id=request.request_id,
        )
        search.assert_current(evidence)
        draft = generator.generate(
            question=request.question,
            blocks=evidence.blocks,
            provider_id=scope.provider_id,
        )
        result = validate_and_render(draft, evidence)
    else:
        result = no_answer(request.request_id, evidence.generation)
    policy.check(
        scope, phase="return", selected_sources=evidence.sources,
        request_id=request.request_id,
    )
    search.assert_current(evidence)
    return result
```

Required collaborator implementations before this code can ship:

Collaborator	Required behavior; not an optional placeholder
`policy.resolve/check`	Fixed-origin authenticated callbacks, bounded response, fresh subject/ethics/policy checks, exact request/scope binding, stage ordering, no expired grants.
`search.retrieve_authorized`	PostgreSQL snapshot, full readiness check, mandatory eligibility in both branches, genuine metadata, bounded context, no text-only provenance loss.
`search.assert_current`	Fresh short read outside the prior transaction; compare durable generation/profile and reject changes.
`generator.generate`	Approved provider mapping, request/context/output bounds, no tools, safe timeouts, structured output, no arbitrary provider URL.
`validate_and_render`	Claim/CTX/quote/source validation below; safe abstention; no external-knowledge escape.
`no_answer`	Typed empty-evidence response; still subject to final authorization checks.

These collaborators are specified seams to implement and test, **not working existing helpers**. The fragment is not a drop-in implementation. Keep route handlers thin and give each collaborator one responsibility.

5.3 Why not call today's `generate_answer()` directly?

The legacy pipeline converts candidates to text, deduplicates by text, uses bracket reference labels, and has external-response/synthesis exceptions. It has no ConPro dispatch callback or structured source ledger.

An integration implementation must preserve typed provenance through retrieval and context selection and insert policy checks before any provider call. Extract narrowly reusable preparation/provider helpers where useful; do not replace the existing default pipeline or change legacy grounding behavior silently. Pin legacy behavior with regression tests.

5.4 Structured answer and evidence contract - proposed

Provider output should be strict JSON containing either `no_answer` or bounded claims. Initial recommendation: at most eight claims, each with a text, one prompt-local `context_id`, and an exact supporting `quote` of at most 800 characters. Compound claims needing multiple sources should be split.

The application, not the provider, assigns stable source descriptors:

```
{
  "contract_version": "conpro-rag.v1",
  "request_id": "<request-uuid>",
  "status": "answered",
  "corpus_generation": 17,
  "retrieval_mode": "hybrid",
  "claims": [
    {
      "text": "<validated statement>",
      "citation_ids": ["C1"]
    }
  ],
  "sources": [
    {
      "citation_id": "C1",
      "document_id": 42,
      "version_id": 108,
      "chunk_id": "<genuine-chunk-uuid>",
      "content_sha256": "<64-lowercase-hex-characters>",
      "projection_version": "document-text-v1",
      "page": null,
      "locator_kind": "section",
      "locator": "Section 2"
    }
  ]
}
```

This JSON is a **shape illustration**; placeholders are not valid production values. `no_answer` has empty claims/sources. Every source in a successful response must be cited; every cited source must be present, eligible, and selected. Model-provided URLs, filenames-as-paths, scores-as-confidence, and unsupported source/CTX identifiers must be rejected, not patched into plausible references.

Critical pure validation fragment:

```
def validate_claims(draft, ledger, support_check):
    accepted = []
    for claim in draft.claims:
        block = ledger.get(claim.context_id)
        if block is None:
            raise InvalidEvidence("Unknown context.")
        if not claim.quote.strip() or claim.quote not in block.text:
            raise InvalidEvidence("Supporting quote does not match.")
        if not support_check(claim.text, block.text):
            raise InvalidEvidence("Unsupported statement.")
        accepted.append((claim.text, block.source, block.locator))
    if draft.status == "answered" and not accepted:
        raise InvalidEvidence("An answer requires evidence.")
    return accepted
```

`InvalidEvidence`, draft models, bounded field checks, and rendering are proposed implementation work. Membership and exact quotes are mechanical checks; **they do not prove that a claim follows from a quote**. Existing token-overlap gates are also not semantic proof. Review representative claims manually and use adversarial grounding tests before approving generative output.

Reject the whole invalid draft for the pilot instead of silently dropping contradictory claims and calling the remainder complete. A structured provider failure returns `provider_unavailable`; do not silently inherit the legacy provider-error answer and call it a successful ConPro generation.

5.5 Protected source navigation: an important browser bottleneck

A plain `<a href="/api/.../download/">` does not attach the JWT held by ConPro's JavaScript client. A direct link to `Document.download` also follows the current pointer rather than the cited version. Use the existing authenticated, version-specific helper instead.

Proposed UI integration fragment, using an **existing** ConPro API helper:

```
import { api } from "./api";

export async function downloadEvidence(source) {
  if (!Number.isSafeInteger(source.version_id) || source.version_id <= 0) {
    throw new Error("Invalid evidence version.");
  }
  const suffix = { pdf: ".pdf", docx: ".docx", txt: ".txt" }[source.format];
  if (!suffix) throw new Error("Unsupported evidence format.");
  await api.documentVersions.download(
    source.version_id,
    `conpro-evidence-v${source.version_id}${suffix}`
  );
}
```

ConPro adds the allowlisted `format` field after reading freshly authorized version metadata; it is not supplied by the provider. Do not insert an arbitrary `href` or use public share tokens. The protected download rechecks access at click time; a 404 after revocation is expected. Use ConPro's existing error handling and refresh logic rather than creating a second token store.

Display document number, title, exact version, source date, and "AI assistance, verify against the official record." Preserve existing layout and existing executive-ask/scratch AI behavior. Never render unsanitized model HTML.

6. Authorized retrieval and hybrid prerequisites

6.1 Integration storage and indexes - proposed

Keep integration evidence in dedicated, access-restricted RAG tables or schema. Do not mix it with legacy unclassified/default-scope chunks.

Proposed durable entity	Minimum responsibilities
`rag_project_state`	`(namespace, project_id)` key; monotonic committed generation; supported active profile.
`rag_source`	Namespace/project/document/version/hash/projection/profile; source operation sequence; publication/readiness/tombstone status; unique immutable source key.

Proposed durable entity	Minimum responsibilities
`rag_chunk`	Genuine UUID; source FK; ordinal; text; 384-dimension vector for the pinned current model; lexical vector; real optional locators.
`rag_operation`	Client/namespace-bound operation ID; idempotency digest; sequence; durable status; safe error code; attempt/timestamps.

Enforce uniqueness/FKs at the database level, positive sequences/ordinals, valid hash lengths, and uniqueness of chunk ordinal per immutable source. The PostgreSQL role must not read ConPro business tables or unrelated RAG schemas. Use parameterized SQL, validated fixed identifiers, and reviewed migrations. An operator must provision the extension with appropriate privileges.

Do not add an HNSW index and assume authorization is solved. For the bounded pilot, prefer an exact vector scan over a materialized authorized subset. An ANN index may internally visit rows beyond a SQL predicate; authorization, recall, side-channel and performance implications need a separate review before switching to shared-index ANN.

6.2 Eligibility SQL - proposed reference fragment

The following CTE is shared by both branches. Table/column names are proposed. The `jsonb` parameter is built from the **verified callback manifest**, never directly from a browser or arbitrary incoming metadata.

```
WITH allowed AS MATERIALIZED (
    SELECT *
    FROM jsonb_to_recordset(%s::jsonb) AS x(
        document_id bigint,
        version_id bigint,
        content_sha256 text,
        projection_version text,
        index_profile text
    )
), eligible AS MATERIALIZED (
    SELECT c.chunk_id, c.text, c.embedding, c.lexical_vector,
           c.page, c.section_label, s.source_pk
    FROM rag_source s
    JOIN allowed a
      ON a.document_id = s.document_id
     AND a.version_id = s.version_id
     AND a.content_sha256 = s.content_sha256
     AND a.projection_version = s.projection_version
     AND a.index_profile = s.index_profile
    JOIN rag_chunk c ON c.source_pk = s.source_pk
    WHERE s.namespace = %s
      AND s.project_id = %s
      AND s.state = 'ready'
)
SELECT chunk_id, text, source_pk, page, section_label,
       embedding <-> %s::vector AS dense_distance
FROM eligible
ORDER BY dense_distance, chunk_id
LIMIT %s;
```

Before querying chunks, prove every manifest source matches one ready published source, using the same snapshot. Missing sources, hash/profile mismatch, or unsupported parser capability produce an explicit not-ready/error result. Zero relevant chunks is `no_answer` only after readiness succeeds.

For the lexical branch, reuse the exact `allowed`/`eligible` definitions and use a fixed reviewed configuration, initially `pg_catalog.simple` for construction identifiers and English pilot material:

```
SELECT chunk_id, text, source_pk, page, section_label,
       ts_rank_cd(
           lexical_vector,
           websearch_to_tsquery('pg_catalog.simple', %s)
       ) AS lexical_score
FROM eligible
WHERE lexical_vector @@ websearch_to_tsquery('pg_catalog.simple', %s)
ORDER BY lexical_score DESC, chunk_id
LIMIT %s;
```

This second fragment replaces the dense SELECT after the same CTE; it is not a standalone query. The indexed `to_tsvector` configuration must match the query configuration. Multilingual stemming, punctuation-heavy drawing codes, and OCR token quality require corpus evaluation; PostgreSQL FTS is not BM25.

6.3 Snapshot, fusion, and bounds

- Open a short `REPEATABLE READ` read-only transaction; capture project generation, verify readiness, run both branches, and copy typed results before committing. Use the same backend authority, connection snapshot, profile, and allowlist.
- Each branch initially returns at most 15 candidates. Union at most 30; final context at most five. Read actual configured bounds and return capability errors on mismatches. Add an explicit total context-character/token budget.
- Fuse by genuine identity, not text: namespace + profile + chunk UUID. Recommended initial RRF is equal-weight `sum(1 / (60 + rank))`, rank starts at one. Treat scores as ranking diagnostics, never confidence probabilities.
- Preserve fused ordering; do not run the legacy lexical reranker as a new primary sort that cancels dense contribution.
- Preserve different records with identical text as separate evidence.
- Statement/query timeouts, maximum source/chunk counts, and connection lifetime need explicit configuration. Proposed pilot ceilings: 200 source references, 10,000 eligible chunks, 24,000 context characters, five context blocks. Exceeding a ceiling is an explicit request/capability error, not truncation disguised as full coverage. Token budgets must also be measured per model.
- After ending retrieval, fresh generation checks reject concurrent publication. This is intentionally conservative: even an unrelated project-corpus change may force a retry. Reduce unnecessary invalidations only with proven source-level freshness semantics.

The existing RAG `read_snapshot()` protects local revision/cache retrieval in one process; it is **not** a durable multi-process corpus generation. The existing mirrored `HybridVectorStore` is still not hybrid retrieval.

6.4 Prompt and external-processing safety

Treat document text as untrusted data, including instructions such as “ignore permissions” or “send this file elsewhere.” Integration v1 has no tools, URL fetching, SQL execution, workflow writes, or cross-project follow-up.

Use a single explicitly approved processor for the pilot. The legacy `auto` mode sends work to two providers and is not an acceptable default for ConPro. Do not silently change the standalone RAG app’s provider setting; resolve the integration processor separately against a server-owned allowlist.

Review processor retention, model training use, residency, contractual terms, and allowed classification before either document export or query context dispatch. Embeddings, lexical tokens, snippets and answers are derived sensitive data, not harmless technical artifacts.

There is an unavoidable boundary: permission checks cannot recall text already sent to a provider. A revocation immediately after dispatch can suppress the answer, but cannot undo that disclosure. The authorization decision is valid at the checked dispatch instant, not a promise of instantaneous distributed revocation.

7. Synchronization, ordering, and atomic publication

7.1 ConPro durable state - proposed migrations

Proposed model	Important fields and constraints
`RagSourceState`	Deployment/project/document; monotonic `change_seq`; desired current version/hash/eligibility; unique logical source key.
`RagOutboxEvent`	UUID; source FK; change sequence; operation kind; safe source manifest; pending/leased/acked/dead state; attempts; lease expiry; unique source+sequence.
`RagSyncReceipt`	Event/operation ID; acknowledged sequence/hash/profile; durable ready status; sanitized error; timestamps.
`RagQueryScope`	Handle digest; subject; request UUID; question digest; source manifest fingerprint; processor/profile/mode; expiry; phase.

Retain immutable logical source keys independently of live business-row FKs. For example, a document pointer may use `SET_NULL`, while the logical document identity and pending tombstone survive deletion. Outbox/source history must not disappear through a document or folder cascade.

Do not retain raw blobs, questions, provider prompts, or source excerpts in the outbox by default. Hashes/identifiers still require restricted access. Decide retention and encrypted storage/backups with the data owner.

7.2 The transactional-outbox rule

Persist source-state change and outbox event **inside the same database transaction as the document mutation**. An `on_commit()` callback may wake a worker, but is not the durable event store.

Proposed transactional fragment:

```
from django.db import transaction


@transaction.atomic
def change_document_and_schedule(document_id, mutate, make_manifest):
    document = Document.objects.select_for_update().get(pk=document_id)
    mutate(document)  # existing authorized domain mutation, not browser code
    document.refresh_from_db()
    state, _ = RagSourceState.objects.get_or_create(document=document)
    state.change_seq += 1
    state.save(update_fields=["change_seq"])
    manifest = make_manifest(document, state.change_seq)
    RagOutboxEvent.objects.create(
        source=state, change_seq=state.change_seq,
        operation=manifest.operation, manifest=manifest.metadata_only(),
    )
```

`RagSourceState`, `RagOutboxEvent`, and manifest functions are proposed, not existing models. All changes to that logical document must take the same parent lock; add database uniqueness and retry tests for first-state creation. Large-file extraction, hashing, network calls, and embeddings must not occur while this transaction/lock is held. Store immutable version identity, then hash the blob in the worker and verify identity again before publication.

Do not wrap arbitrary user callbacks or create a general mutation framework in production. Apply this transaction pattern to named existing domain-service entry points. Review all of them:

Mutation path	Required handling
Initial upload / new version	Advance sequence and index the exact immutable version; new revision may reset eligibility.
Review/issue/approve/reject/withdraw	Re-evaluate current-source eligibility, sequence, and tombstone as required.
Confidentiality/date changes	Deny current queries immediately through policy; durable replace/tombstone follows.
Document/folder delete	Capture tombstones before cascades remove identifying rows.
Administrative imports/bulk updates	Use the same domain service or explicit outbox writes; `QuerySet.update()` bypasses model save signals.
Membership/role/ethics/processor change	Live query recheck takes effect without waiting for indexing; revoke affected handles and reconcile export eligibility.

Existing version creation/status updates span multiple statements. Do not assume they already constitute this transaction. Signals alone do not cover every bulk or administrative path. Inventory paths and add tests before claiming complete event coverage.

7.3 Worker and delivery semantics

Use a bounded management command/worker in the existing deployment:

1. Claim due events with `select_for_update(skip_locked=True)`, set a short lease, commit the claim transaction, then perform external work.
2. Reload the source; check sequence, approved indexing scope, processor/storage policy, immutable version, size/type, and blob existence.
3. Stream a file through private staging; verify actual bytes and content digest. Reject redirects/arbitrary destinations and decompression/parser abuse.
4. Submit with a deterministic idempotency key and bounded timeouts.
5. Poll a **durable** RAG operation record; a request timeout is not proof of failure. Query the same operation before resubmitting.
6. On `ready`, persist the exact sequence/hash/profile receipt. Acknowledge only the matching event; do not mark a newer desired state ready.
7. Retry transient failures with capped exponential backoff and jitter; proposed initial policy is five attempts, 2-second base, 60-second cap, then dead-letter. Operator replay preserves the original logical operation identity.
8. Recover expired leases and periodically reconcile the approved source manifest.

Use at-least-once delivery plus idempotency; do not promise exactly-once network delivery. Worker restart, duplicate response, timeouts after commit, and lost acknowledgement must not create duplicate chunks or roll back source ordering.

7.4 Safe RAG publication

Idempotency digest includes namespace, logical document, source sequence, operation, version/content hash, projection, and index profile. Ingress verifies namespace ownership and the latest ConPro source-operation eligibility. Staging is not searchable.

After extraction/embedding succeeds:

```
BEGIN
  lock rag_project_state(namespace, project_id)
  lock logical-source operation state
  if sequence < last_seen: acknowledge stale operation without activation
  if sequence = last_seen and digest differs: reject integrity conflict
  if exact operation already committed: return its existing receipt
  verify publication preconditions and immutable staged manifest
  publish source metadata + dense vectors + lexical entries
  apply old-version deactivation/tombstone for this logical document
  increment project generation once
  mark durable operation ready/committed
COMMIT
```

Dense/lexical/source/generation publication must be one PostgreSQL transaction, not two store writes followed by an optimistic ready flag. Failed publication leaves previous committed data intact, but ConPro's live policy decides whether that older version is still eligible. A pending new revision is not permission to serve the old one as current.

Ordering guards protect against **received** stale operations. They cannot know about an event not yet delivered. The latest-source check plus live query policy protects that gap; reconciliation provides eventual convergence.

No query transaction spans this publication, callback, or AI call. Avoid distributed transactions across the two application databases; recover with durable receipts and idempotent replay.

Protect version blobs against in-place modification. The existing version model's append-only intention is not proof of immutable filesystem/object storage. Pin the worker-verified digest in ConPro's durable source receipt, restrict blob writes, and verify storage versioning/integrity during reconciliation. If immutable storage cannot be guaranteed, agree a rehash/serving policy before claiming that a version ID always identifies identical bytes. Never silently reuse a stored digest for a changed blob.

7.5 Parsing and provenance bottlenecks

The existing extractor returns flattened text. It does not preserve PDF page provenance through chunking. Before promising page citations, introduce an additive structured extraction result for this integration and test it:

```
ExtractedBlock(text, page_or_none, section_or_none, parser_version)
    -> chunk with source identity + exact inherited locator
    -> UUID/ordinal + text hash + vector + lexical representation
```

Never infer a page number from `Section N` or from a model-generated reference. Until page-aware extraction is approved, use real section/offset locators and `page=null`. Scanned PDFs may have no extractable text; DOCX tables, complex tables, drawings, CAD/BIM geometry, OCR language packs, and embedded objects need explicit coverage decisions. No text yields `unsupported` or a sanitized extraction failure, not ready.

Recommended pilot file cap: 20 MiB even though the standalone RAG API permits larger files. Also cap extracted characters, pages, chunks, parser runtime, temporary storage, and concurrent extraction. Choose and test concrete parser limits before enabling files beyond the controlled pilot.

For untrusted files in production, run parsing in a constrained worker process with no unnecessary network access and enforce memory/CPU/time limits. Merely placing a large parsing task in a Python thread is not resource isolation.

7.6 Deletion, revocation, retention, and reconciliation

Tombstone source eligibility immediately in ConPro. Queries then fail closed through scope rechecks even if RAG deletion is delayed. Apply physical removal to text, vectors, lexical data, staged files, operations, and any approved caches under the agreed retention policy. Prevent stale replay from resurrecting it.

Reconciliation compares desired ConPro source keys/sequences/hashes against RAG receipts, repairs missing/extra/stale entries, and raises alerts. It must not re-admit withdrawn/ineligible content just because the blob still exists.

Deletion from active indexes is not deletion from backups or a provider's prior processing. Record those retention/disclosure limits explicitly. Default reconciliation target for review: every 15 minutes; it is a recovery target, not the authorization enforcement mechanism.

7.7 Extending the foundation across every ConPro module

The document-version contracts/SQL in Sections 4-6 are the first implementable slice, not a universal representation for PMIS records. Do not put RFIs, payment rows or daily reports into fabricated `DocumentVersion` records.

For each additional source kind:

1. The owning module defines an explicit projection and lifecycle eligibility, reusing its actual list/detail/object/field permissions.
2. Capture an immutable authorized projection snapshot with a monotonic source revision in the same outbox scheme. A mutable row plus `updated_at` alone is not a reliable historical evidence version.
3. Add a discriminated source contract and storage mapping. Negotiate the new contract version before routing that source kind; old v1 consumers reject unknown kinds rather than interpreting them as documents.
4. Apply that same policy/projection at indexing, scope issuance, dispatch, return and source navigation. Any loss of field access invalidates the whole broader projection unless a separately versioned narrower projection is available.
5. Add module-specific tombstones, record/attachment events, projection changes, audit, source actions and acceptance tests.

Recommended first record extension, **proposed contract v2**:

```
class RecordProjectionRef(BaseModel):
    model_config = ConfigDict(extra="forbid", frozen=True)
    kind: Literal["record_projection"] = "record_projection"
    model_label: Literal["rfis.rfi"]
    record_id: str = Field(min_length=1, max_length=128)
    source_revision: int = Field(strict=True, gt=0)
    projection_version: Literal["rfi-response-v1"]
    content_sha256: str = Field(pattern=r"^[a-f0-9]{64}$")
    index_profile: str = Field(min_length=1, max_length=80)
```

This fragment uses the imports from Section 4.4. It is an explicit first extension, not an unrestricted Django model name supplied by a client. Add reviewed source types deliberately. Generic storage may use a typed `source_kind` and internal source UUID, but each kind still needs a fixed authorization adapter and validated natural key.

7.8 Critical safe RFI projection - proposed reference fragment

The following uses verified RFI fields and existing role/field helpers. `authorize_processing` is a proposed server-owned approval function, not a callback supplied by the user. `source_revision` comes from durable source state. The caller must first load the row through the same scoped record queryset as the existing RFI interface.

```
from rest_framework.exceptions import PermissionDenied

from apps.accounts.role_policy import allows, module_for
from apps.accounts.role_records import field_allowed
from apps.accounts.scoping import require_project_access
from apps.rfis.models import RfiStage


def project_rfi(user, rfi, source_revision, authorize_processing):
    if not user.is_authenticated or not user.is_active:
        raise PermissionDenied("Projection unavailable.")
    require_project_access(user, rfi.project_id)
    if source_revision <= 0:
        raise ValueError("A durable source revision is required.")
    if rfi.stage not in (RfiStage.RESPONDED, RfiStage.CLOSED):
        raise PermissionDenied("Projection unavailable.")
    fields = ("number", "subject", "question", "answer", "stage")
    module = module_for(type(rfi), "rfis")
    if user.role_catalog_enabled and not all(
        allows(user, rfi.project_id, module, action, rfi)
        for action in ("view", "export")
    ):
        raise PermissionDenied("Projection unavailable.")
    if not all(
        field_allowed(user, rfi, name, action=action)
        for name in fields for action in ("view", "export")
    ):
        raise PermissionDenied("Projection unavailable.")
    authorize_processing(
        user=user, record=rfi, projection="rfi-response-v1", fields=fields
    )
    return {
        "kind": "record_projection",
        "model_label": "rfis.rfi",
        "record_id": str(rfi.pk),
        "source_revision": source_revision,
        "projection_version": "rfi-response-v1",
        "fields": {name: getattr(rfi, name) for name in fields},
    }
```

Hash the canonical serialized projection, pin its schema/profile and preserve it with the source revision. Never use `model_to_dict`, unrestricted serializers, `SELECT *`, or a full `custom`/attachment payload as an indexing shortcut. Free-text questions/answers may themselves contain confidential or personal content: a harmless-looking field name does not establish processor approval.

The reference denies a projection if any included field is unauthorized. It does **not** index a privileged full projection and then hide individual fields after retrieval. Apply this same rule to submittals, commercial narratives, daily reports and quality records. Reuse actual module-specific visibility; do not copy the RFI stage or document approval rules to unrelated models.

7.9 Live operational facts and cross-module answers

Current numbers belong to a separate typed, read-only facts path inside ConPro. Each approved facts operation must:

- Apply current project/module/record/field authorization before aggregation.
- Call the existing calculation/reporting logic or a separately reviewed deterministic service, not generated SQL or model arithmetic.
- Return a factual receipt with operation/version, project, `as_of`, data revision, explicit units/currency, authorized source descriptors, and approved display values/rounding. Use decimal values, not floating-point money.
- Enforce bounds, timeouts and processor approval. Sensitive figures may be displayed deterministically without sending them to an external model.
- Recheck policy/freshness before release. If a consistent multi-module snapshot is unavailable, label each fact's actual timestamp and do not claim a common transaction snapshot.

Proposed factual receipt illustration:

```
{
  "kind": "live_fact",
  "operation": "<approved-server-owned-report-operation>",
  "operation_version": "1",
  "project_id": 7,
  "as_of": "<timezone-aware-timestamp>",
  "facts_revision": "<authoritative-revision>",
  "metric_id": "<approved-metric>",
  "decimal_value": "125000.00",
  "currency": "<project-currency>",
  "display_value": "<ConPro-formatted-value>",
  "source_ids": ["<authorized-report-or-record-reference>"]
}
```

This is not an existing endpoint or a real financial result. The operation catalog, facts schema and module adapters require implementation approval. The generated explanation must not change numbers, units, dates, rounding or sign. Validate numerical statements against typed receipts and retain the existing executive facts/reference safeguards.

For mixed questions, present clearly separated current facts and document/ record evidence. For example, explain an authorized schedule issue using a current schedule receipt plus linked RFI/specification excerpts. If the user cannot see the RFI, do not expose its title, snippet or existence through the schedule answer. Unsupported questions abstain; no unrestricted autonomous cross-module investigation is enabled.

7.10 System-wide UI and human control

Use one contextual AI capability through existing project and module screens, with the current project/module/record supplied as context hints, not authority. Server-side checks still decide the accessible evidence. A project-level entry can offer cross-module answers; module-level entry points can narrow context. Do not create another login or redesign the application around a chatbot.

Suggested later drafting can help prepare an RFI response, report narrative, submittal comment or action checklist. It must be visibly marked as a draft, retain evidence and original record revision, and require a human to use the existing form/workflow with normal validation and permissions. No automatic submission, approval, certification, payment, incident closure or schedule baseline change is included. Any write automation requires its own proposal.

8. HTTP client, errors, limits, and capacity

8.1 Fixed-origin ConPro gateway - proposed reference fragment

Suggested home: `ConPro\backend\apps\rag\client.py`. Inputs below come from validated settings and server-owned state, not the browser.

```python
import json

import requests


def call_rag(base_url, credential, payload):
    try:
        with requests.post(
            base_url + "/integrations/conpro/v1/ask",
            json=payload,
            headers={"Authorization": f"Bearer {credential}"},
            timeout=(3, 35),
            allow_redirects=False,
            stream=True,
        ) as response:
            if response.status_code != 200:
                raise map_rag_status(response.status_code)
            body = bytearray()
            for part in response.iter_content(chunk_size=8192):
                body.extend(part)
                if len(body) > 262144:
                    raise GatewayProtocolError("Response exceeds limit.")
    except requests.Timeout as exc:
        raise GatewayTimeout("RAG request timed out.") from exc
    except requests.RequestException as exc:
        raise GatewayUnavailable("RAG connection unavailable.") from exc
    try:
        return IntegrationAnswer.model_validate(json.loads(body))
    except (ValueError, UnicodeDecodeError) as exc:
        raise GatewayProtocolError("Invalid RAG response.") from exc
```

The error classes, `IntegrationAnswer`, and `map_rag_status` are proposed typed adapter code. Startup must validate an HTTPS fixed base origin, no credentials in the URL, no query/fragment, no arbitrary path, and a server-approved host. Normalize a trailing slash once at startup. Never use `verify=False` or follow credential-bearing redirects. Do not log chained exception text/headers/payloads; log a safe error code, exception class, and request UUID.

A network read timeout is not a strict total wall-clock deadline. Add a monotonic request budget across callbacks/retrieval/generation, check it between phases, and refuse expired work. Proposed initial target: 40 seconds overall, 3-second connect timeout, 20-second provider network timeout, and bounded policy callbacks. Measure before treating these as achievable SLOs. Remote work may continue after a caller times out; do not automatically repeat billable inference.

8.2 Error mapping - proposed public contract

Condition	ConPro HTTP / safe code	Engineering behavior
Feature off / invisible project or source	404 / `not_found`	Preserve non-disclosure; no provider call.
Valid visible request violates explicit integration policy	403 / `not_permitted`	No source details outside permitted scope.

Condition	ConPro HTTP / safe code	Engineering behavior
Missing/invalid/oversized query scope	400 or 422 / `invalid_request` or `scope_too_large`	Correct request or narrow explicitly; never silently truncate.
Eligible source not indexed at required version/hash/profile	409 / `index_not_ready`	Show authorized readiness; queue/recover indexing separately.
Scope/version/generation changes during query	409 / `scope_stale`	Discard draft; bounded explicit retry with a new scope.
Source file unsupported / file too large	422 / `unsupported_source`; 413 / `source_too_large`	Keep a durable sanitized status; no ready flag.
Invalid evidence/provider response contract	502 / `invalid_evidence`	No invented citation or raw model output.
RAG internal credential rejection or bad protocol	502 / `integration_unavailable`	Alert operators; **do not return upstream 401 as a ConPro login failure**.
Capacity limit / temporarily unavailable dependency	503 / `busy` or `dependency_unavailable`	Safe `Retry-After`; bounded backoff only where appropriate.
Expired request budget / network timeout	504 / `request_timeout`	No guaranteed cancellation; inspect operation receipt before ingestion retry.
Valid retrieval without sufficient evidence	200 / status `no_answer`	Empty citations; no claim that the project has no such information.

The gateway must map 3xx, malformed JSON, unknown contract versions, missing fields, unknown status codes, and oversized responses explicitly. Do not forward arbitrary service response bodies to the UI.

8.3 Concurrency and scaling

Reuse RAG's current nonblocking four-question default as a **shared process-wide budget**. Release slots in `finally`, including permission rejection, malformed provider output and dependency errors. Keep active-index transactions short. Give policy callbacks/ordinary ConPro requests headroom so answer saturation cannot starve the checks needed to finish those same answers.

Do not increase FastAPI API workers: the current standalone ingestion queue and other state remain process-local. Integration durable tables do not magically make the rest of the application multi-worker safe. Scale instances only after all relevant shared state and admission/provider quotas are addressed.

The earlier 3.1x throughput improvement used a simulated provider and is not a ConPro capacity promise. This document does not set a maximum real-user count. First establish a pilot envelope: 20 selected documents, four concurrent questions, real representative file sizes, measured token costs, and provider quotas. Then agree a questions-per-second/p95/error budget target.

9. Audit, deployment, rollout, and recovery

9.1 Minimal audit events

Reuse ConPro's existing `record_event` helper. Proposed event types include `RAG_QUERY_COMPLETED`, `RAG_QUERY_DENIED`, `RAG_SYNC_READY`, `RAG_SYNC_FAILED`, and `RAG_SOURCE_REVOKED`; register them in the actual audit catalog/UI where required instead of assuming free strings are fully supported.

Record actor via existing mechanisms, authorized project, request/operation UUID, mode/profile, safe result code, elapsed time, and source count. Do not record question/answer text, snippets, embeddings, credentials, scope handles, DSNs, provider headers, raw exception messages, or public share links.

Recommended pilot policy: successful answers require a durable minimal audit event before ConPro releases them. An audit write failure returns an explicit unavailable response; do not claim an audited success. Review this availability tradeoff with the owner.

9.2 Configuration inventory - all integration settings proposed

Setting/decision	Required behavior
`CONPRO_RAG_ENABLED`	False by default; hides UI entry and rejects API/scope issuance when disabled.
Namespace + project/document selection	Server-owned deployment mapping; never derived from incoming email/project code.
RAG base origin / ConPro callback origin	Fixed validated private HTTPS origins; explicit certificate trust.
Query/ingestion/callback credentials	Separate secret-manager entries, rotation and least privilege; no `VITE_*` secrets.
Approved processor policy	Per project/classification; no default `auto`/unapproved provider.
Contract/index profile/retrieval mode	Explicit compatible versions; mismatch fails readiness.
Source/scope/context/parser limits	Conservative pilot caps; startup validates positive values.
Durable outbox/operation retention	Reviewed retention, lease/backoff/reconciliation and dead-letter ownership.
Database credentials	RAG-only derived-data role; ConPro-only authoritative role; no shared writable superuser credentials.

Keep the real personal-key RAG policy and existing internal retrieval/inference credentials separate. Never put ConPro users into a privileged shared demo account to avoid implementing the actual policy boundary.

9.3 Delivery batches and ownership

Batch	Small reviewable scope	Owner / stop condition
I0 - approval	Approve scope, processor, identity mapping, status/date policy, SLO/data retention.	Product + document controller + security/operations; no runtime code beforehand.
I1 - contracts and policy	Add disabled adapter shell, shared document visibility extraction, scope models/handlers, strict schemas.	ConPro/RAG backend engineers; policy parity tests required.
I2 - durable indexing	Outbox/worker, immutable manifests, constrained extraction, source/chunk/operation tables, atomic publication.	Backend/data engineers; replay/crash/delete tests required.
I3 - scoped dense query	Exact authorized retrieval, structured evidence, dispatch/return checks, shared admission, audit.	RAG + ConPro engineers; no browser rollout until isolation tests pass.

Batch	Small reviewable scope	Owner / stop condition
H1 - hybrid dependency	Independent lexical branch, shared snapshot, bounded RRF, parity and quality evaluation.	RAG engineers; separate hybrid approval.
I4 - UI pilot	Existing ConPro request/download helpers, authorized readiness, safe errors/citations, feature-off parity.	Frontend + QA; document-controller acceptance.
I5 - operational release	Disposable-backend tests, representative load, restore/reconcile, quotas/alerts/rollback drill.	Operations + QA + owner signoff.
S1 - workflow knowledge	RFI, submittal and change narratives with module-owned versioned projections and authorized cross-links.	Module owners + backend/QA; foundation I1-I3 and each module's policy/lifecycle tests.
S2 - live project controls	Approved project/contract, payment/cash-flow, schedule and reporting fact adapters; mixed evidence/fact answers.	Commercial/schedule/reporting owners; deterministic numeric parity and freshness gate.
S3 - field and quality operations	Daily reports, QC findings, eligible incident/NCR projections and parent-controlled attachments.	Field/quality/safety owners; company, draft, duty and restricted-field gates.
S4 - supporting modules	Approved spreadsheet/custom-field projections and governance guidance; optional Scratch-pad consumer.	Data/module owners; schema, privacy and evidence-action gates.
S5 - reviewed drafting	Suggested narratives/comments/actions through existing module forms.	Explicit separate approval; no autonomous business writes.

Review each batch independently. Optional dense-only I4 requires an explicit decision to defer H1; a "hybrid integration" release requires H1 to pass. I0-I5 establish the shared foundation; they are **not completion of the system-wide objective**. S1-S4 deliver the broader PMIS coverage and may progress in independently approved module batches once their shared prerequisites pass. Scratch-pad is not a prerequisite for any other module. No calendar or staffing estimate is reliable until I0 decisions and corpus coverage are known.

9.4 Deployment and rollback sequence

1. Back up both databases using existing procedures; verify a restore to an isolated environment. Inventory derived-data retention and key access.
2. Deploy additive migrations with both integration flags off. Do not grant public reachability to private integration/callback endpoints.
3. Validate service identities, credential rotation, TLS, fixed origins, capabilities and profile compatibility without indexing real project files.
4. Index synthetic approved sources in staging, complete the acceptance matrix, and rehearse crash recovery and denial paths.
5. Obtain processor/data-owner approval, then index only the selected pilot sources. Compare hashes/readiness/locators manually with ConPro.
6. Enable only for named pilot users/project. Monitor callbacks, outbox age, denied/stale queries, provider errors/cost, p95, capacity rejections and audit.
7. Expand only after signed acceptance and measured resource/permission parity.

Rollback: disable ConPro issuance/UI first; reject integration queries at RAG; let bounded in-flight work settle or expire; stop ingestion workers by owned process identity; retain operations for diagnosis. Restore the previous code with additive schema retained unless a tested migration rollback is approved. Do not roll back ConPro business records or remove the standalone RAG features.

A profile rollback must select an explicitly compatible corpus and current source manifest. Never relabel an older index current. Revoke credentials and purge derived data according to policy when retiring the integration.

10. Acceptance tests engineers must implement

No runtime integration tests were executed for this document. These are required future tests, not passing-test claims.

Test group	Minimum executable cases / required outcome
Feature off	Existing ConPro/RAG endpoints and AI hooks unchanged; new public/private routes unavailable.
Authentication	Missing/wrong/rotated service credentials; personal/demo keys rejected by integration; service token rejected as browser identity; TLS/redirect and exact-prefix tests.
Subject lifecycle	Inactive account, expired membership/role, changed ethics acknowledgement, revoked session where supported, expired/replayed/mismatched handle; no disclosure.
Policy parity	Legacy and published-role modes; program/admin/project oversight, external-company rules, confidential sources, export denied, per-version object checks.
Pre-retrieval isolation	Two projects and two users with identical question/vector/text; hidden source must never enter dense/lexical candidate sets, counts, prompts, logs, or output.
Callback denial	Unavailable/stale dispatch check means zero provider calls; return denial suppresses the draft.
Current revision	New draft suppresses old approved version; same-name rename stable; changed status/dates/confidentiality invalidates; approved new version appears only when ready.
Source integrity	Tampered bytes/hash/profile; missing blob; digest reuse with conflicting manifest; unsupported type; oversized stream/zip bomb/parser timeout; no searchable partial publication.
Idempotency/order	Duplicate event, old event after tombstone, same sequence/different digest, timeout after commit, lost acknowledgement, lease recovery, dead-letter replay.
Atomicity	Fail lexical write after dense staging; fail before generation bump/receipt; readers see old coherent publication or new coherent publication, never mixed branches.
Evidence	Unknown CTX, invented page/URL, same text/different document, invalid quote, unsupported claim, numerical contradiction, external-response prefix and prompt injection.
Cache and concurrency	No integration cache bypass; four shared slots across both APIs; fifth request retryable; release on every failure; no index lock during callbacks/provider work.

Test group	Minimum executable cases / required outcome
Downloads and UI	Existing JWT refresh, version-specific blob download, 404 after revocation, safe filenames, no raw HTML, stale response discarded on user/project change.
Recovery/observability	Reconciliation repairs missed events without resurrection; restore drill; audit failure; sanitized logs; alerts for outbox age/callback failures/cost.
Backend/quality/load	Disposable real PostgreSQL/pgvector + FTS; representative documents; approved processor in staging only; measured quality/latency/cost and overload.
System-wide module parity	Each enabled module has approved projections, its own lifecycle rules, record/field/company tests and protected source actions; one document pilot is not accepted as full PMIS coverage.
Live facts / cross-module answers	Exact amount/currency/date/rounding parity with authoritative reports; linked inaccessible records absent; mixed snapshot times labeled; model cannot manufacture operations or SQL.
Draft assistance	Suggestions remain unsaved/unsubmitted until human action through existing forms; optimistic record revision checks and normal business validation still apply.

10.1 Critical denial test shape - proposed

```
def test_revocation_before_dispatch_never_calls_provider(
    integration_client, scope_factory, policy_control, provider_spy
):
    scope = scope_factory(project="project-a")
    policy_control.revoke_after_retrieval(scope.subject_id)
    response = integration_client.ask(scope, "What is required?")
    assert response.status_code in (403, 409)
    provider_spy.assert_not_called()
    assert "sources" not in response.json()
```

The fixtures/spies are proposed test infrastructure. Also test revocation **after** dispatch: no answer returned, but explicitly acknowledge the provider may already have received context.

10.2 Minimum pilot signoff evidence

- Zero unauthorized candidate/prompt/source disclosures in the isolation matrix.
- Zero stale replay resurrection and zero partial-publication visibility under injected failures.
- Every displayed source opens the exact authorized version; unavailable locators remain absent, never invented.
- A reviewed question set covering exact references, paraphrases, revisions, abstention, numerical claims, and malicious document instructions. Record relevant-source recall, citation correctness, faithfulness and abstention errors; agree thresholds before evaluation.
- At least a 15-minute staging run at four concurrent clients with representative documents and quotas; report request counts, p50/p95/p99, errors/503s, callback and database time, token cost, memory and outbox lag. No universal QPS promise.
- Credential rotation, provider outage, worker crash, reconciliation, backup restore, and feature-disable drills demonstrated.

Use mocked providers/models for deterministic unit tests. Use only explicitly designated disposable databases for destructive integration tests. Real-provider tests require approved synthetic or approved project data and explicit cost authorization. Never run them against a live PMIS database.

11. Bottleneck register and engineering resolutions

Bottleneck	Required resolution / accountable gate
Shared project does not mean shared record visibility	Live ConPro policy, exact source-version set before both retrieval branches; policy parity gate I1/I3.
Main RAG personal-key middleware rejects service calls	Exact mounted-boundary authentication wiring; no anonymous prefix; I1.
Generic POST maps to edit permission	Explicit read/export policy in ConPro APIView; retain authentication/ethics; I1.
Source set too large / expensive role queries	Bounded pilot, measured query counts, explicit too-large response; request-local optimization only after equivalence tests.
New file is draft while old index is approved	Suppress old current result; exact source readiness; controller-approved lifecycle; I2/I3.
Existing text pipeline discards provenance	Add typed integration evidence path and locator-aware extraction; do not reuse text dedupe/citations unchanged; I2/I3.
Lexical/dense generation mismatch	Same short PostgreSQL snapshot, atomic publication, profile handshake, no silent fallback; H1.
Missing or delayed revoke/delete event	Live policy rechecks enforce denial; outbox/reconciliation removes derived data; I2/I3.
Timeout after successful ingestion	Durable operation ID/receipt and idempotent replay; never clear/re-add blindly; I2.
Refresh race / service 401 logs out real user	Existing ConPro refresh helper; map internal auth failure to gateway error; I3/I4.
JWT source links fail or open a newer file	Existing authenticated version-specific blob helper; I4.
External processing exceeds view permission	Explicit export/processor policy, single approved processor, no default auto; I0/I3.
Provider outage / malicious or unsupported claims	Explicit unavailable/abstention states, strict evidence mapping and adversarial evaluation; I3/I5.
Load starves policy callbacks	Shared bounded answer admission, callback headroom, ingress controls and measured load; I5.
“Millions of users” assumption	Separate account count, simultaneous clients and questions/second; measure before scaling infrastructure.
PDF/CAD/OCR coverage overstated	Capability matrix and locator truth; unsupported status; parser/resource gate I2.
Version row points at mutated blob	Protected/versioned storage, pinned digest and integrity/reconciliation policy; do not assume model comments enforce immutability.
Scratch-pad becomes the sole AI integration	Shared backend contracts and contextual consumers across project/module screens; S1-S4 coverage gates.

Bottleneck	Required resolution / accountable gate
All PMIS rows treated as documents	Versioned per-module projections and discriminated source contracts; no fabricated DocumentVersion rows.
Stale indexed numbers presented as current	Separate authoritative live-facts path, factual receipts, exact numerical validation and timestamps; S2.
Module field policy lost in shared embeddings	Fixed safe projection; deny broader projection if any field loses authorization; S1-S4.
Rollback resurrects old/withdrawn evidence	Feature disable plus current policy checks; compatible profile rebuild, not stale index relabeling; I5.

12. Manageable implementation layout and coding rules

Suggested files are **proposed**, not created by this document:

Side	Focused home	Responsibility
ConPro	`backend\apps\rag\api.py`, `serializers.py`	Thin HTTP boundaries and strict user inputs.
ConPro	`backend\apps\rag\policy.py`	Existing policy composition, scope issuance/revalidation, processor approval.
ConPro	`backend\apps\rag\client.py`	Fixed-origin service calls, bounded decoding, safe error mapping.
ConPro	`backend\apps\rag\models.py`, migrations	Scope/outbox/source state and receipts; constraints.
ConPro	`backend\apps\rag\sync.py`, management command	One durable worker implementation, replay/reconciliation.
ConPro	Small projection/facts adapters owned by each enabled module	Explicit field/lifecycle policy; reuse shared integration contracts, not a second AI engine per module.
RAG	`app\conpro\api.py`, `contracts.py`	Private child application, authenticated validated contracts.
RAG	`app\conpro\query.py`, `policy_client.py`	Short orchestration and fixed policy callbacks.
RAG	`app\conpro\repository.py`	Scoped SQL, publication, durable operations/generations.
Both	Focused test modules	Policy, lifecycle, contracts, concurrency and failure injection separately.

Do not create these files just to match a diagram. Combine tiny related modules until complexity justifies a split. Keep routes thin; use typed immutable contracts and database constraints; inject only real external boundaries for testing; avoid generic plugin registries, inheritance trees, duplicate policy engines, and catch-all exception handling.

Share the authoritative visibility helper with existing document endpoints; share eligibility SQL between dense/lexical branches; share a single admission budget. Do not copy authorization logic into React, prompts, and SQL independently. Never introduce `as any`, broad untyped payloads at permanent boundaries, or success-shaped fallbacks for broken contracts.

Keep existing Python/JS formatting/testing tools and dependency manifests. Any new library or infrastructure dependency requires an explicit justification, pin/lock update, license review and reproducible installation. Do not replace the current application or redesign the UI to implement this adapter.

13. Assumptions, limitations, and future recommendations

These sections apply to the **entire system-wide integration program**, not only its document pilot. An assumption is not an existing verified capability. If it proves false, revisit the affected design before approving its batch.

13.1 Assumptions and how engineers must verify them

ID	Assumption / planning basis	Required verification or action
A1	The two source baselines in Section 1 are the intended integration targets.	Rebase the implementation plan against actual approved release branches; rerun contract/policy parity tests after upstream changes.
A2	ConPro remains the sole business identity, permission, workflow and calculation authority.	Owner approval; no connector-managed substitute accounts or shadow business logic.
A3	A stable deployment namespace and project mapping can be assigned.	Explicit namespace/company/deployment mapping; test multiple deployments and identical local record IDs.
A4	Private HTTPS endpoints, certificate validation and managed service secrets can be provisioned.	Operations validates network paths, DNS, certificates, ingress limits and rotation; mTLS capability is not assumed to exist today.
A5	Approved PostgreSQL/pgvector/FTS and restricted roles are available for derived evidence.	Disposable-backend compatibility/migration/restore tests; capacity and backup ownership agreed before release.
A6	Every enabled module can produce a bounded safe projection or authoritative facts receipt.	Module owner approves fields, lifecycle, source revisions, processor rights and source navigation; unsupported modules stay disabled.
A7	File versions can be protected against in-place modification and reliably read by the authorized exporter.	Storage versioning/ACL review, digest pinning, missing-blob/integrity tests; adopt a documented rehash policy if necessary.
A8	Data owners can approve processors, residency, retention and permitted classifications.	Written processing decision before real documents/facts leave ConPro; no provider assumed approved from an existing API key.
A9	ConPro policy callbacks remain available during queries.	Measure callback latency/headroom; fail closed during outages rather than reuse stale authorization.
A10	The initial 20-document/four-question pilot is operationally modest.	Measure real parsing/model memory, database time, provider limits and cost; these counts are review defaults, not measured production capacity.

ID	Assumption / planning basis	Required verification or action
A11	Status/date, financial metric and schedule definitions can be agreed with domain owners.	Document controller/commercial/schedule signoff; use configured ConPro timezone initially and explicit units/currency/rounding.
A12	Changes to business mutation paths can support atomic outbox persistence.	Inventory API, workflow, import, admin and cascade paths; test crash windows and bulk-update coverage.
A13	Engineers will implement only specifically approved batches.	Record reviewer, scope and conditions; producing this document or its code fragments is not authorization.

Do not turn an unmet assumption into a silent fallback. Examples: unavailable live policy is not permission to use yesterday's grant; missing hybrid support is not permission to label dense retrieval hybrid; missing current facts are not permission to estimate a payment total from indexed text.

13.2 Current and inherent limitations

ID	Limitation	Consequence / mitigation
L1	The ConPro integration is not implemented or tested end-to-end.	All proposed routes/models/code seams require implementation and acceptance tests; no production-readiness claim.
L2	System-wide module coverage is a staged objective, not a feature already available.	Publish a per-module readiness register; document pilot completion is not PMIS-wide completion.
L3	Document parsing, OCR, tables, drawings and multilingual retrieval have incomplete coverage.	Declare unsupported cases, test real samples, preserve locator truth and expand only through reviewed parser/profile changes.
L4	Citations, quotes and lexical grounding do not prove semantic truth or completeness.	Human validation, adversarial/representative evaluation and abstention; users verify official records.
L5	Permission checks cannot undo provider disclosure already made.	Check immediately before dispatch, minimize content, approve processors and suppress results after revocation; document the residual race.
L6	Scope and generation checks are not a distributed transaction across ConPro, RAG and a provider.	Conservative stale-result rejection, idempotency, durable receipts and reconciliation; no instant global consistency claim.
L7	Indexes and projections lag business changes.	Live authorization/freshness checks gate queries; live facts path handles current figures; never label pending/stale evidence current.
L8	The standalone RAG API still has process-local state and bounded single-process admission.	Do not add workers/instances blindly; four active answers is not support for millions of simultaneous users.

ID	Limitation	Consequence / mitigation
L9	Exact authorized-subset search and repeated policy checks can become expensive.	Bounded scopes, measured query counts and load; review optimized authorization-preserving search before expanding.
L10	Network timeouts do not guarantee end-to-end cancellation or prevent duplicate provider billing.	Request budgets, operation receipts, explicit retry rules and cost controls; no blind inference retry loop.
L11	Storage/provider backups and retention may outlive active-index deletion.	Approved deletion/retention schedules and operator evidence; do not promise immediate physical erasure everywhere.
L12	Current plan does not certify contractual, legal, safety, engineering or financial decisions.	Assistance only; existing responsible people and ConPro workflows retain authority.
L13	Deployment topology, actual quotas, recovery SLOs and production policies have not been verified.	Complete I0/I5 operational gates with real approved environments; do not infer them from local code.

Privacy/security, authorization and numerical correctness are release gates, not limitations that can be waived implicitly to obtain a faster demonstration.

13.3 Future feature recommendations and sequencing

Horizon / recommendation	Intended enhancement	Prerequisite and guardrail
Next - system-wide S1-S4 coverage	Shared contextual assistance across workflows, project controls, field/quality operations, reporting and supporting modules.	Approved module projections/facts adapters and policy parity; not a Scratch-pad-only rollout.
Next - measured hybrid retrieval	Better exact reference/code matching plus semantic questions.	Independent lexical branch, identical eligibility, consistent publication, evaluated RRF; no silent fallback.
Next - evaluation and feedback loop	Curated module-specific questions, citation/faithfulness/abstention metrics and permitted user feedback.	Restricted retention and owner-approved evaluation data; feedback never overrides authorization or official records.
Next - richer locators and source previews	Page/section/table-cell evidence and exact version preview within existing screens.	Parser-derived locations, safe rendering and authenticated downloads; no fabricated pages or public blob links.
Medium - revision comparison	Explain differences between two explicitly selected authorized versions.	Historical-source contracts, both-version permissions, status labels and comparison tests; do not confuse history with current guidance.
Medium - multilingual/OCR/table support	Broader project language/file coverage.	Representative corpora, language-specific evaluation, resource limits and new index profiles/reindexing.
Medium - approved live-fact expansion	More current reporting, schedule and commercial explanations.	Deterministic calculation adapters, typed values, snapshot/freshness receipts and numerical parity tests.

Horizon / recommendation	Intended enhancement	Prerequisite and guardrail
Medium - human-reviewed drafting	Suggested RFI responses, report narratives, comments and action checklists.	Separate approval; visible draft status, evidence and existing form validation; no autonomous submission or approval.
Medium - persistent authorized caching	Reduce repeated retrieval/provider cost.	Shared generation and full policy/source/provider-aware key/invalidation design; revalidate every hit and permission change.
Medium - scalable durable processing	Larger corpora and more concurrent users through shared workers/instances.	Externalize all relevant process state, quotas, load/soak tests, backup/recovery and cost budgets first.
Later - bounded investigation	Multi-step, permission-aware follow-up within approved read-only module/fact operations.	Explicit tool catalog, step/time/cost caps, per-step authorization, audit and safe stop; no arbitrary SQL, URLs or business writes.
Later - operational self-healing	Retry known transient failures, resume leases, reconcile indexes and execute approved recovery runbooks.	Observable conditions, tightly scoped permissions, attempt limits and rollback; human approval for code/schema/permission/financial changes.
Later - portfolio and multi-region	Authorized cross-project analysis and reduced geographic delay.	Separate portfolio rights, data-residency decisions, consistency design, measured demand and tested regional recovery.
Conditional - graph/link enhancements	Better explanation of existing authorized business relationships.	Start with explicit ConPro links; add a graph store only if measured queries justify its complexity and access model.

These are recommendations, not approved backlog commitments, installed capabilities or promises of autonomy. Prioritize demonstrated user value, correctness, security, operational cost and maintainability over adding infrastructure or “agentic” features.

14. Review decisions and approval checklist

Resolve these before approving I1:

Decision	Proposed recommendation / required reviewer
Pilot project/documents/users	One project, at most 20 normal eligible documents; owner/document controller selects.
System-wide coverage and module order	Approve S1-S4 roadmap and name each module owner; document pilot is the first stage, not the final scope.
Record projections and live facts	Approve per-module schemas/processor rights and authoritative metric sources before indexing or exposing them.
Namespace/company mapping	Stable deployment namespace; do not assume org hierarchy equals tenant; application owner.
Export and processor policy	Explicit approval beyond view permission; security/data owner.

Decision	Proposed recommendation / required reviewer
Status/date interpretation	`APPROVED` and `ISSUED_FOR_CONSTRUCTION`, date rules above; document controller.
Dense-only or hybrid pilot	Explicit dense-only approval or H1 prerequisite; engineering + owner.
Hosting/TLS/residency	Private origins, trusted certificates, approved provider/location; operations/security.
Retention/audit failure policy	Minimal required audit, scoped retention/backups, provider disclosure terms; owner/security.
Service identity/rotation	Registered least-privilege clients, mTLS where supported, secret manager; operations.
Parser coverage/resource limits	Real representative files and verified locators; engineering + document controller.
SLO/cost/rollout target	Measured four-client pilot first, agreed latency/error/token budgets; owner/operations.

Review fields:

- [ ] Architecture and dependency boundaries accepted.
- [ ] System-wide objective, module roadmap and owners accepted; not Scratch-pad-only.
- [ ] Identity/policy/processor model accepted.
- [ ] Document status/revision/date/coverage rules accepted.
- [ ] Proposed contracts and critical reference code reviewed.
- [ ] Durable synchronization, atomicity, deletion and rollback accepted.
- [ ] Assumptions validated or assigned owners; limitations and residual risks accepted.
- [ ] Future recommendations prioritized separately from the approved implementation scope.
- [ ] Test matrix, staging data, cost and SLO targets agreed.
- [ ] Specific implementation batch authorized: _____
- [ ] Reviewer / date / conditions: _____

Until a batch is explicitly approved, this remains a document. No integration route, model, migration, worker, UI hook or runtime configuration is authorized.

Appendix A. Verified source map

These references identify existing behavior, not proposed functions. `ConPro\` denotes the separate ConPro repository; `app\` denotes the RAG repository. Line numbers apply to the source baselines in Section 1.

Existing behavior	Verified reference
ConPro route prefixes and existing authentication routes	`ConPro\backend\config\urls.py:8-29`
Default audited JWT/session, ethics and scoping filters	`ConPro\backend\config\settings.py:138-174`
Active/project/hierarchy scoping and project 404	`ConPro\backend\apps\accounts\scoping.py:24-115`
DRF request/object permission composition	`ConPro\backend\apps\accounts\permissions.py:50-184`
POST/custom action mapping, including export	`ConPro\backend\apps\accounts\role_policy.py:57-85`

Existing behavior	Verified reference
Model-to-module lookup and published-role filters/checks	`ConPro\backend\apps\accounts\role_policy.py:382-383,417-479`
Record and field-specific restrictions	`ConPro\backend\apps\accounts\role_records.py:131-176`
Actual status/confidentiality constants	`ConPro\backend\apps\documents\models.py:134-154`
Current pointer, dates, document and version identity	`ConPro\backend\apps\documents\models.py:168-276`
Existing visibility helper to extract without broadening	`ConPro\backend\apps\documents\views.py:31-60`
Uploading a new revision changes current pointer/control	`ConPro\backend\apps\documents\views.py:180-215`
Protected exact-version download view and router	`ConPro\backend\apps\documents\views.py:418-436`; `ConPro\backend\apps\documents\urls.py:1-14`
Existing JWT refresh and generic API request helper	`ConPro\frontend\src\api.js:1-120`
Existing authenticated exact-version download helper	`ConPro\frontend\src\api.js:449-452`
Existing executive facts/numeric/reference behavior to preserve	`ConPro\backend\apps\reporting\executive.py:506-541`
Existing RFI fields/stages and scoped workflow interface	`ConPro\backend\apps\rfis\models.py:33-48,100-170`; `ConPro\backend\apps\rfis\views.py:16-73`
Parent-specific attachment visibility	`ConPro\backend\apps\attachments\views.py:20-126`
Incident row/duty/restricted-section separation	`ConPro\backend\apps\incidents\access.py:1-84`
Existing semantic audit helper	`ConPro\backend\apps\audit\models.py:355-399`
RAG current personal/service auth separation	`app\auth.py:115-197`
RAG public equality-filter request and text answer shape	`app\schemas.py:19-46`
RAG pre-body middleware and global dependencies	`app\main.py:101-106`; `app\auth.py:146-166`
RAG process-wide current answer limiter/return recheck	`app\main.py:125-131,484-525`
Local vector read/replace locking, not a distributed generation	`app\vector_store.py:581-636`
Text-only dedupe/rerank loses source associations	`app\rag.py:480-516`
Existing scoped generation path to preserve by default	`app\rag.py:945-1040`
Current supported suffixes and flattened PDF extraction	`app\ingestion.py:32-90`

Appendix B. Deliverable and verification scope

This review package is intended as identical Markdown/PDF copies in both repositories' `docs` folders. Code samples are reference fragments with explicitly named implementation seams. Syntax checks and source-reference checks do not mean the proposed integration runs, passes tests, or is production-ready.

No ConPro or RAG application source, database, dependency, credential, permission, running service, or deployment is changed by preparing this document. The document does not authorize implementation or commit/push of application changes.